

Building APIs with R plumber2

Lecture 15

Dr. Colin Rundel



plumber2

plumber and plumber2 are R packages that allow you to create RESTful APIs using specially annotated R scripts.

Both packages make use of a Roxygen-like syntax to provide annotations (decorators) to R functions that then become the API endpoints with specific parameter, behaviors, etc.

plumber2 is a complete rewrite of plumber and it provides a more modern and flexible interface, improved performance, and additional features. The APIs between the two packages are not compatible.

Like shiny - the package handles all the necessary processing as well as the launching of the necessary web server to host the API.

A basic endpoint

A plumber2 API is just an R script that defines one or more of R functions that have specifically formatted comments that define the endpoint behavior.

Annotations are specified using `##` comments directly above the function definition. Annotation keywords usign the `@` symbol.

For example the script below defines a single endpoint that responds to `GET` requests at the `/hello` path and returns the string “hello world”. Note the

```
1 ## Return "hello world"
2 ## @get /hello
3 function() {
4   "hello world"
5 }
```

Running your API

To run a plumber API you read the script in using the `api()` function to create a server object.

The API can then be launched using `api_run()` and stopped using `api_stop()`. When used interactively (i.e. in RStudio) the API will run in the background and not block your session.

Both the host/ip and port can be configured via `api()` or `api_run()`.

One important note is that plumber2 is relatively new and does not have full integration into RStudio yet and in most cases the IDE will get confused and assume your script is a plumber API - make sure to check what is actually running!

Handler methods

When defining an endpoint you must specify the HTTP method(s) that the endpoint will respond to. As we just saw this is done using `@+method` keywords in your annotations.

Each of your plumber2 endpoints can support one or more of the following verbs:

- `@get`
- `@put`
- `@head`
- `@post`
- `@delete`
- `@any`

```
1  #* @get /cars
2  function() {
3      ...
4  }
5
6  #* @post /cars
7  function() {
8      ...
9  }
10
11 #* @put /cars
12 function() {
13     ...
14 }
```

```
1  #* @get /cars
2  #* @post /cars
3  #* @put /cars
4  function() {
5      ...
6  }
```

Endpoint inputs

When making requests to an API endpoint, data can be passed in a number of different ways:

- Via the path - e.g. `@get /user/<id>`
- via a query string - e.g. `@get /user?id=1231`
- or via the body of the request - e.g. `POST` or `PUT` requests

Path parameters

Path parameters are indicated in the method annotations using `<>` around the parameter name.

Each parameter specified in the path must have a corresponding argument in the function definition.

While not required, it is good practice to include `@param` annotations for each parameter to document their purpose.

```
1  /** Echo the parameter that was sent in
2  /**
3  /** @get /echo/<msg>
4  /**
5  /** @param msg:string The message to echo back.
6  /**
7  function(msg) {
8      list(
9          msg = paste0("The message is: '", msg, "'")
10     )
11 }
```

Path priority

While this does not typically come up, it is helpful to understand how plumber2 handles potentially ambiguous paths.

When multiple endpoints could match a given request path, plumber2 uses the following priority rules to determine which endpoint to invoke:

1. `/path/to/something/specific`
2. `/path/to/<name>/specific`
3. `/path/to/<name>/<setting>`
4. `/path/to/something/*`
5. `/path/*`

Query parameters

For endpoints users may pass in additional arguments using query strings, e.g. `?q=bread&pretty=1` at the end of the URL.

plumber2 will automatically parse these into a named list which can be accessed via the reserved `query` argument in the endpoint function.

Similar to path parameters additional documentation on query parameters can be provided via `@query` annotations.

```
1  ## Fake search query endpoint ala DuckDuckGo
2  ##
3  ## @get /
4  ##
5  ## @query q:string The search query
6  ## @query pretty:int Whether to pretty-print the results (1) or not (0)
7  ##
8  function(query) {
9      paste0("The q parameter is '", query$q %||% "", "'. ",
10           "The pretty parameter is '", query$pretty %||% 0, "'.")
11 }
```

Type hints

Path and query parameters are transmitted as strings by default, if you are expecting a different type you can specify it using a type hint in the `@param` or `@query` annotation.

The following types are supported by default:

R Type	Plumber Name
<code>logical</code>	boolean
<code>numeric</code>	number
<code>integer</code>	integer
<code>character</code>	string
<code>Date</code>	date
<code>POSIXlt</code>	date-time
<code>raw</code>	byte, binary
<code>vector</code>	<code>[...]</code>
<code>list</code>	<code>object({name:Type, ...})</code>

Request body

Both path and query parameters are passed in via the URL - this works for relatively small amounts of data but is not a good option for larger payloads.

Larger data can be passed in the body of the request - this is typically used with **PUT**, **POST**, and **PATCH** requests. The size limit varies a lot depending on the API server configuration but is typically around ~2MB by default.

By default plumber2 will attempt to parse the body based on the **Content-Type** header of the request. Further control can be achieved using the **@parser** annotation - there are a large number of built-in parsers available and custom parsers can also be defined.

The result of parsing the body is made available to the endpoint function via the reserved **body** argument.

Serialization

By default, the return value of a plumber2 endpoint is serialized to JSON (via `jsonlite`). This can be overridden by using the `@serializer` annotation.

Some of the available serializers,

Annotation	Content Type	Description/References
<code>@serializer html</code>	<code>text/html; charset=UTF-8</code>	Passes response through without
<code>@serializer json</code>	<code>application/json</code>	Object processed with <code>jsonlite::toJSON()</code>
<code>@serializer csv</code>	<code>text/csv</code>	Object processed with <code>readr::format_csv()</code>
<code>@serializer text</code>	<code>text/plain</code>	Text output processed by <code>as.character()</code>
<code>@serializer jpeg</code>	<code>image/jpeg</code>	Images created with <code>jpeg()</code>
<code>@serializer png</code>	<code>image/png</code>	Images created with <code>png()</code>
<code>@serializer svg</code>	<code>image/svg</code>	Images created with <code>svg()</code>
<code>@serializer pdf</code>	<code>application/pdf</code>	PDF File created with <code>pdf()</code>
<code>@serializer rds</code>	<code>application/rds</code>	Object processed with <code>base::serialize()</code>
<code>@serializer parquet</code>	<code>application/parquet</code>	Object processed with <code>nanoparquet::write_parquet()</code>

Plot output

```
1  #* Plot out data from the palmer penguins dataset
2  #*
3  #* @get /plot
4  #*
5  #* @query spec:string If provided, filter the data to only this species
6  #* (e.g. 'Adelie')
7  #*
8  #* @serializer png
9  #*
10 function(query) {
11     myData <- penguins
12     title <- "All Species"
13
14     # Filter if the species was specified
15     if (!is.null(query$spec)){
16         title <- paste0("Only the '", query$spec, "' Species")
17         myData <- subset(myData, species == query$spec)
18     }
19
20     plot(
```

request

One additional reserved argument available to plumber2 endpoint functions is `request`. This is a `reqres` object that provides access to the full HTTP request details as an R6 object.

From this it is possible to get access to: cookies, headers, raw query string, raw body content, and much more.

response & server

Similar to `request`, the `response` reserved argument provides access to a `response` object that represents the HTTP response.

Normally, `plumber2` will construct this object for you, but if necessary you can modify it directly to set custom headers, cookies, status codes, and body content.

Finally, the `server` reserved argument provides access to the underlying `plumber2` API server object itself. This is useful for advanced use cases.

Errors

Plumber2 wraps all endpoints to handle errors that occur during execution.

```
1  ## Example of throwing an error
2  ## @get /simple
3  function() {
4    stop("I'm an error!")
5  }
```

By default these errors will return a generic 500 Internal Server Error response to the user. However, plumber2 (and reqres) provides a number of helper functions to generate more friendly error messages with appropriate HTTP status codes.

```
1  ## Generate a friendly error
2  ## @get /friendly
3  function() {
4    abort_bad_request(
5      "Your request could not be parsed"
6    )
7  }
```

Live Demo